PAIRED IMAGE DATASET CREATION TO APPLY A VINTAGE DIGITAL CAMERA LOOK TO IPHONE IMAGES

A THESIS SUBMITTED TO

THE FACULTY OF ARCHITECTURE AND ENGINEERING

OF

EPOKA UNIVERSITY

BY

HALIL BAGOSI

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR BACHELOR DEGREE

IN

SOFTWARE ENGINEERING

JUNE 2026

I hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all material and results that are not original to this work.

Name Surname: Halil Bagosi

Signature: _____

ABSTRACT

PAIRED IMAGE DATASET CREATION TO APPLY A VINTAGE DIGITAL CAMERA LOOK TO IPHONE IMAGES

Bagosi, Halil
B.Sc., Department of Computer Engineering
Supervisor: M.Sc Edlira Cani

The artificial processing of smartphone images was first introduced as a way to improve blurry and unimpressive shots from small camera sensors. This artificial processing has become crucial to the production of a smartphone photo, giving rise to the term 'Computational Photography'. Computational Photography helped perfect smartphone images to have great HDR and bring detail even to dark, undetailed parts of an image. Recently, the general public has grown tired of this artificial perfect photo and is seeking out the imperfect look from older vintage digital cameras. This thesis presents the creation and preprocessing of a paired image dataset captured synchronously using an iPhone 17 Pro Max and a Fujifilm JZ100. The methodology for capturing 61 image pairs and the creation of a robust preprocessing pipeline to handle the complex spatial misalignments caused by optical parallax are detailed in this thesis. The pipeline includes global alignment via SIFT and RANSAC, dense local alignment using optical flow, and rigorous patch extraction filtering. This work provides a structured foundation for training deep convolutional models to map smartphone image characteristics to DSLR-quality outputs.

Keywords: *Deep Learning, Image Enhancement, UNet, Dataset Creation, Image Registration, SIFT, Digital Camera*

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my supervisor for their invaluable guidance throughout this research. I also thank Epoka University for providing the academic environment to pursue this project.

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
DPED	DSLR Photo Enhancement Dataset
DSLR	Digital Single-Lens Reflex
FOV	Field of View
GAN	Generative Adversarial Network
ISP	Image Signal Processor
PSNR	Peak Signal-to-Noise Ratio
LPIPS	Learned Perceptual Image Patch Similarity
RANSAC	Random Sample Consensus
ResNet	Residual Neural Network
AI	Artificial Intelligence
SIFT	Scale-Invariant Feature Transform
SSIM	Structural Similarity Index
CMOS	Complementary Metal-Oxide-Semiconductor
CCD	Charge-Coupled Device
LUT	Look-Up Table
ARM	Advanced RISC Machines
HDR	High Dynamic Range
CPU	Central Processing Unit
GIL	Global Interpreter Lock
I/O	Input/Output
JPEG	Joint Photographic Experts Group
PNG	Portable Network Graphics
DNG	Digital Negative
SSD	Solid State Drive
RAM	Random Access Memory
NVMe	Non-Volatile Memory Express
OOM	Out Of Memory
ReLU	Rectified Linear Unit
DoG	Difference of Gaussians
BSI	Back-Side Illuminated
SoC	System on Chip
UMA	Unified Memory Architecture
PCIe	Peripheral Component Interconnect Express
MPS	Metal Performance Shaders

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The evolution of computational photography over the past decade has been exponential, taking full advantage of tiny smartphone camera sensors and processing to produce high-fidelity results. Smartphone cameras, such as the Apple iPhone 17 Pro Max, featuring an $f/1.78$ main CMOS sensor combined with a neural engine, can produce seriously impressive results. These modern devices use multi-frame noise reduction, semantic segmentation, and aggressive tone mapping to synthesize photos which is usually called Computational Photoghraphy, that may beat out dedicated DSLRs in tricky lighting.

Physics is important in photography though, which means that sensor size, focal length and light distance and sensitivity matter and cannot always be reproduced artificially. This way of processing has trended toward hyper-realism and clinical sharpness, since smartphone cameras used to produce less detailed, blurry photos.

Nowadays, smartphones produce those sharp images, but that is not what the general public is interested in anymore. There has been a massive shift in what people appreciate, along with the rise of Artificial Intelligence, people are moving away from perfection and appreciating the parts of art that make it human and inherently imperfect.

That's why there has been a resurgence of small vintage digital cameras. An example is Fujifilm JZ100. The JZ100 relies on an aging 14-megapixel Charge-Coupled Device (CCD) sensor. By modern standards, its dynamic range is terrible, it crushes shadows and it blows out bright skies instantly. Yet, it possesses a distinct, almost film-like color science and a beautiful, organic noise profile that modern algorithms try (and usually fail) to replicate with cheap filters.

This interesting phenomena of balancing hardware and software capabilities is the main motivation of this thesis. The main focus of this thesis is to attempt to create a paired dataset to see if we could force a deep neural network to learn the highly complex, non-linear mapping required to translate a "perfect" computational CMOS image into the heavily degraded, distinct domain of a legacy CCD sensor.

1.2 Problem Statement

The main problem we tackle in this thesis is the uniformity between the image pairs for the dataset. This dataset aims to align shots between an Apple iPhone 17 Pro Max and a Fujifilm JZ100 so that a neural network can learn to apply the imperfections and color science of the Fuji camera to the iPhone.

Evidently, carrying this out required solving multiple physical hardware and software problems:

- **Noise Profiles:** CCD sensors don't produce uniform static. We had to teach the network to replicate stochastic Poisson and Gaussian noise distributions that shift depending on the luminance of the scene.
- **Color Science:** The Bayer filter array on a 2012 compact camera processes light entirely differently than an Apple sensor. Complex tensor transformations were needed for this thesis, not simple 3D LUTs.
- **Dynamic Range and Clipping:** Handling this was the most meticulous part of crafting the dataset. The Fujifilm JZ100 has a very low full-well capacity. Often times, images of bright areas captured from the sensor, due to minimal processing, turned out as pure white $(255, 255, 255)$. The models had to learn how to aggressively destroy data, forcing the wide-dynamic-range iPhone image into a crushed CCD color space.

1.3 Core Research Objective

The primary goal of this thesis is to architect and evaluate a robust deep learning pipeline capable of this specific sensor domain translation. Finding out if convolutional architectures could genuinely simulate the hardware limitations and aesthetic footprint of a legacy CCD sensor using only a pristine CMOS ground truth was the main objective.

These were the major steps taken to carry on this thesis:

1. Building a strictly controlled, stereoscopic dataset of perfectly aligned image pairs shot simultaneously in the real world.
2. Engineering a heavily optimized, parallel data pipeline capable of preprocessing uncompressed 4K imagery on an ARM architecture, breaking the Python Global Interpreter Lock (GIL) on an Apple M1 Pro.

3. Testing whether a purely mathematical model (U-Net) or an adversarial model (Pix2Pix) was actually better suited for the job, forcing us to confront the classic Perception-Distortion tradeoff.

1.4 Engineering Challenges and Scope

This project faced challenges both in the physical creation of the dataset and the software implementation of the models. After capturing 61 total pairs of images, the size of the input folder reached 417 megabytes. Processing these high resolution images required implementing strategic optimizations.

To get around the bottleneck, a custom Hybrid Parallel Pipeline had to be implemented for the M1 Pro workstation. Utilizing Apple's Unified Memory Architecture, it made possible isolating the heavy mathematical alignment math (SIFT/RANSAC) to individual processes, while offloading the massive disk-writing operations to a sprawling thread pool. The AI models were also trained on the same system using MPS on Apple M1 Pro.

1.5 Thesis Structure

This thesis was organized as such:

- **Chapter 1: Introduction** goes into the shift in the perception of photography and the problem statement of this thesis, showing what this thesis is really about.
- **Chapter 2: Literature Review** dives into the math behind image alignment and the theoretical differences between regressive networks and GANs.
- **Chapter 3: Methodology** breaks down the process of creating the dataset and capturing the images, including using Adobe Project Indigo to bypass the iPhone's aggressive ISP and HDR processing.
- **Chapter 4: Implementation** details the techniques and technologies used to implement the solution to the problem presented in this thesis.
- **Chapter 5: Results** moves the focus to the RTX 3090, detailing the loss convergence and how Pix2Pix ultimately outclassed U-Net in perceptual realism.

- **Chapter 6: Conclusion and Discussion** concludes the findings of this thesis and outlines future work to further the solution to this problem.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to Domain Translation

The main focus of an Image-to-Image (I2I) domain translation is figuring out how to turn one type of image into another while keeping the underlying structure intact. The foundational math relies on the assumption that an image from a source domain $\mathscr{X}$ can be deterministically or probabilistically mapped to a target domain $\mathscr{Y}$. The source domain in this case is the computationally processed Apple iPhone 17 Pro Max image from its CMOS sensor, whereas the target domain, $\mathscr{Y}$, is the muddy, dynamic-range-crushed, and heavily textured space of a 2012 Fujifilm JZ100 CCD sensor.

Early attempts to emulate old cameras leaned heavily on handcrafted 3D Look-Up Tables (LUTs) and simple film grain overlays. LUTs are great for fast, mathematically exact color grading, but are completely blind to spatial geometry. A LUT applies the exact same color shift to a pixel regardless of where it lives on the sensor. It doesn't take into account lens vignetting, chromatic aberration, or the physical structure of blown-out highlights. To truly translate the physical geometry of a sensor—not just its color—we knew we had to step away from traditional signal processing and lean into deep convolutional networks.

2.2 Images

In the digital world, pictures are binary representations of visual data [1]. They are generally categorized into two types:

Raster Images: Made up of a grid of tiny colored squares called pixels. Photos taken with digital cameras are raster images.

Vector Images: Use mathematical formulas to draw lines, curves, and shapes. These can be scaled to any size without losing quality and are commonly used for logos and illustrations [2].

Digital images produced by cameras are raster images. These images are a 2D

grid of pixels structured in rows and columns [3]. The Coordinate Grid is tied to the image resolution e.g a 1920x1080 image contains 2,073,600 individual pixels arranged in 1,080 rows and 1,920 columns [4]. Each one of these pixels store a composite value of three primary color channels(Red, Green, Blue).

The amount of unique colors an image can display is measured by its bit-depth. For standard 8-bit per channel images, the possible unique colors it can display can be found by multiplying the possible values per channel, 256(0-255) possible values stored per 8-bit channel, which leads to $256 \times 256 \times 256 = 16.7$ million possible unique colors. [5] [6]

For normal use these images need to be compressed using a certain method. These include JPEG, DNG, PNG and such. This dataset consists of JPEG images. JPEG is a widely adopted approach to lossy compression for digital images, especially those from digital photography. Users can modify the compression level, enabling a flexible balance between file size and image quality.

JPEG commonly reaches 10:1 compression, accepting a noticeable yet broadly acceptable decline in image quality. [7]

Since its 1992 debut, JPEG has remained the world's most prevalent image compression standard, [8] [9] and the leading digital image format, generating several billion JPEG images daily as of 2015. This extensive use and user familiarity guided its selection for constructing the dataset.

2.2.1 Global Adjustments

A global adjustment is an image processing operation that applies a uniform mathematical function to every single pixel in the entire image grid indiscriminately, regardless of its position or surrounding pixels. In computer science, global adjustments are classified as Point Operations—where an output pixel depends solely on its input value, independent of the surrounding pixels [5].

Exposure and Brightness

To globally brighten an image, a software algorithm takes the RGB values of every single pixel and multiplies them by a constant scaling factor (e.g., $NewValue = OldValue \times 1.2$). To darken, it divides them. If any value exceeds 255 during this multiplication, it is truncated to 255, a phenomenon known as highlight clipping or blowing out the highlights.

Contrast

Global contrast adjustments use a midpoint value (typically 128 in an 8-bit image) as an anchor. The algorithm pushes pixel values that are already above 128 closer to 255 (making bright areas brighter) and pushes values below 128 closer to 0 (making dark areas darker). This widens the total dynamic spread between the lightest and darkest parts of the frame.

Saturation

Saturation measures the intensity of a color relative to its own brightness. The algorithm calculates the grayscale (luminance) value of a pixel first. To increase saturation, it mathematically pulls the individual R, G, and B values further away from that grayscale average. For example, if a pixel is muted red (150, 100, 100), increasing saturation pushes the red channel higher while pulling green and blue lower, making it $(180, 70, 70)$.

White Balance

Global color temperature shifts work by applying different multipliers to specific color channels while leaving others alone. To make an image look "warmer," the algorithm multiplies all Red values by a factor greater than 1 (e.g., 1.1) and all Blue values by a factor less than 1 (e.g., 0.9).

2.3 Image Analysis

Image analysis extracts meaningful information from images—primarily digital ones—using digital image processing methods. Such tasks range from reading barcoded tags to identifying a person by their face.

2.4 Deep Learning Architectures for Translation

2.4.1 U-Net

U-Net is an advanced convolutional neural network created for image segmentation. [10] Built on a fully convolutional neural network, [11] its design was adjusted and broadened to function with fewer training images while producing more accurate segmentation. Processing a 512×512 image requires under a second on a contemporary (2015) GPU when using the U-Net framework. [10] [12] [13] [14]

This architecture includes a contracting path and an expansive path forming its

u-shaped design. The contracting path follows a standard convolutional model, using repeated convolutions, a rectified linear unit (ReLU), and max pooling; as contraction progresses, spatial detail declines while feature detail grows. The expansive pathway combines feature and spatial information through up-convolutions and concatenations with high-resolution data from the contracting path. [15]

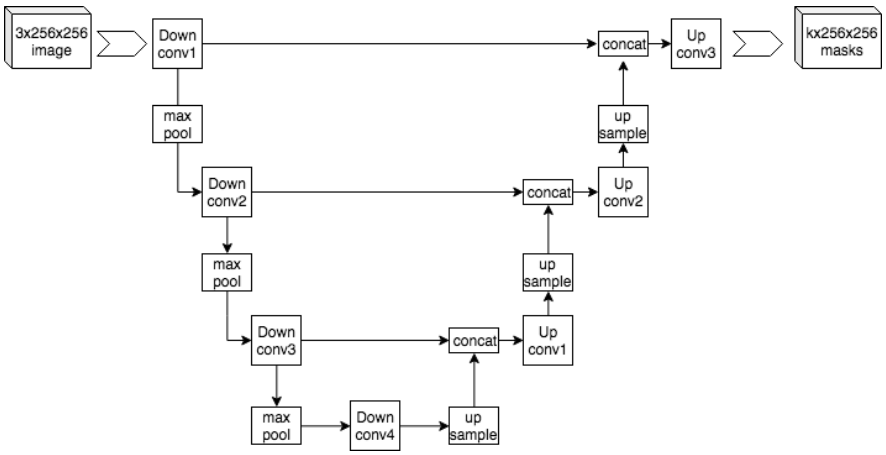

Figure 2.1. U-Net architecture for producing k 256-by-256 image masks for a 256-by-256 RGB image.

2.4.2 Conditional GANs and Pix2Pix

A generative adversarial network (GAN) is a type of machine learning framework and a fundamental technique in generative artificial intelligence. [16] In a GAN setup, two neural networks compete in a zero-sum game, where the gain of one directly translates into the loss of the other.

By leveraging training data, GANs acquire the ability to produce new outputs that follow the patterns of the originals. For instance, a GAN trained on photographs can produce new images that seem realistically convincing to human viewers. Although GANs were first created as generative models for unsupervised learning, [17] they have since become valuable for semi-supervised learning, [18] supervised work, [19] and even reinforcement learning situations.

The mechanism behind GANs relies on indirect feedback from the discriminator, a network that assesses how authentic an input seems. [20] This interaction encourages the generator to produce examples that fool the discriminator rather than simply minimizing a distance metric to a real image. This approach supports effective unsupervised learning.

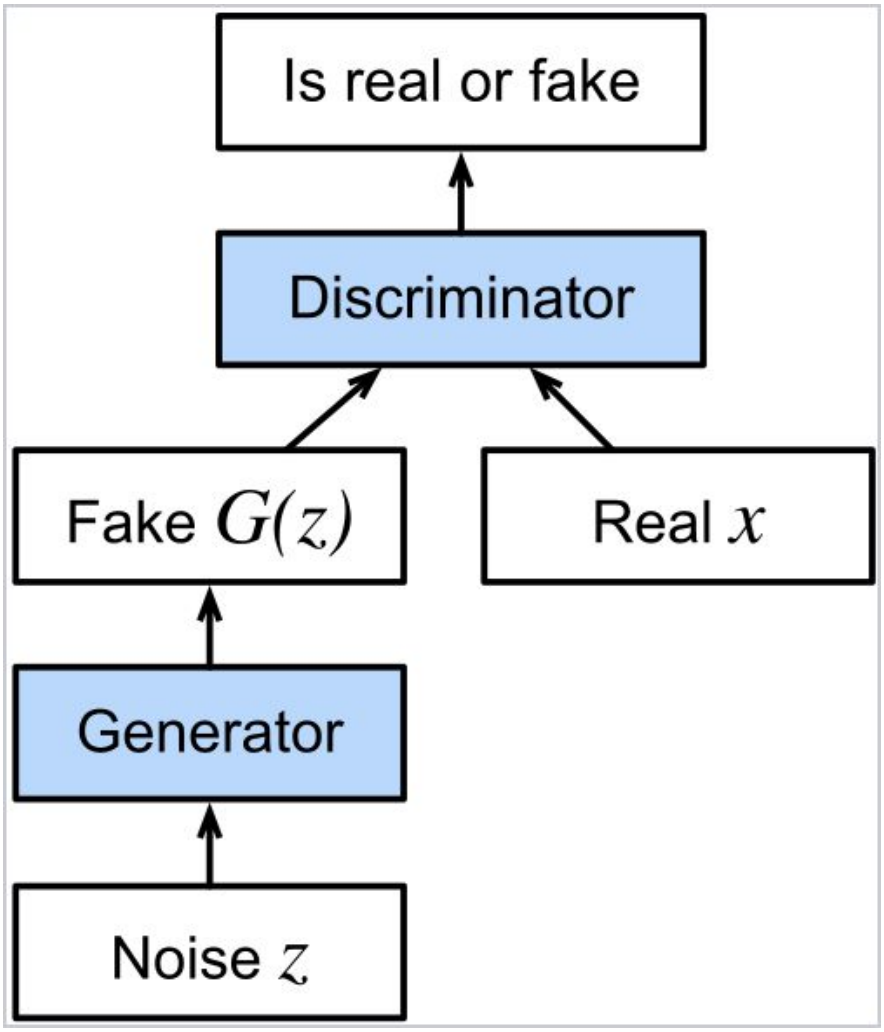

Figure 2.2. An illustration of how a GAN works

Pix2pix is a deep learning model for paired image-to-image translation. Introduced by Phillip Isola et al. in 2016 and presented at CVPR 2017, it uses conditional Generative Adversarial Networks (cGANs) to map an input image to an output image, such as turning a sketch into a photo or a map into an aerial view. [21]

Instead of predicting pixels from scratch, it learns the mapping between two different visual depictions of the same scene. [22] The model requires a dataset of paired images (e.g., input "A" is a label map, output "B" is the corresponding real photo) to learn properly. [21]

It utilizes a U-Net architecture to pass information through skip connections from early encoding layers directly to later decoding layers. This preserves fine spatial details that standard encoders/decoders typically lose. [23] [21]Unlike standard discriminators that evaluate an entire image as "real" or "fake", the PatchGAN classifies small N × N patches. This enforces sharp, high-frequency localized details while using

fewer parameters and computing faster. [24]

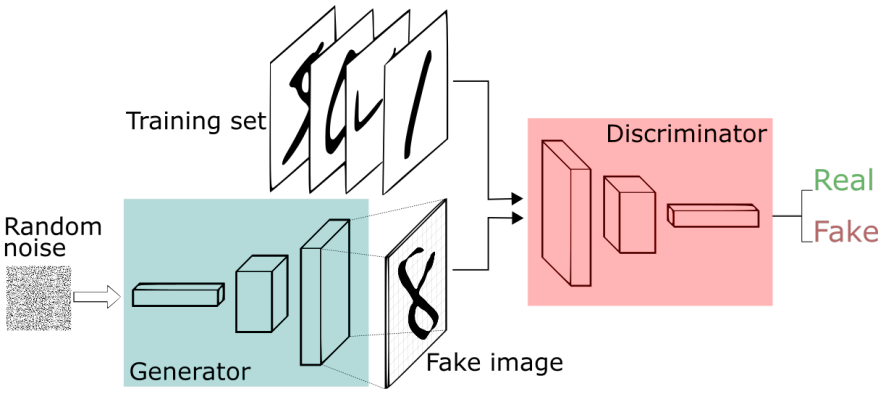

Figure 2.3. An illustration of how Pix2Pix works

2.4.3 Evaluation Metrics

1.PSNR

Peak signal-to-noise ratio (PSNR) is a professional term describing the relationship between a signal's maximum power and the disruptive noise that reduces its accuracy. Since many signals span a broad dynamic range, PSNR is typically shown as a logarithmic value on the decibel scale. [25]

PSNR often measures reconstruction quality for images and videos affected by lossy compression. Here, it serves as a full reference metric, contrasting a noisy version with the original noise-free source. [25]

PSNR is most clearly explained through mean squared error (MSE). For a noise-free $m \times n$ monochrome image I and its noisy counterpart K, MSE is defined as:

$$\mathrm{MSE} = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} [I(i,j) - K(i,j)]^2 \tag{2.1}$$

The PSNR (in dB) is defined as:

$$\begin{aligned} \text{PSNR} &= 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right) \\ &= 20 \cdot \log_{10} \left(\frac{\text{MAX}_I}{\sqrt{\text{MSE}}} \right) \\ &= 20 \cdot \log_{10}(\text{MAX}_I) - 10 \cdot \log_{10}(\text{MSE}) \end{aligned} \tag{2.2}$$

MAX_IMAX_I denotes an image's highest possible pixel value. With 8 bits per sample, this figure is 255. More broadly, for linear PCM using B bits per sample, $\text{MAX}_I = 2^B - 1$. **2.SSIM**

The Structural Similarity Index Measure (SSIM) is a sophisticated technique used to evaluate the perceived quality of digital images and videos, including television and cinematic content. It functions as a full-reference metric, requiring an initial distortion-free or original image to serve as the baseline for quality assessment. SSIM operates by analyzing the similarity between two images based on three key components: luminance, contrast, and structural information. These comparison functions form the foundation of the SSIM calculation. [26]

$$l(x,y) = \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1} \tag{2.3}$$

$$c(x,y) = \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2} \tag{2.4}$$

$$s(x,y) = \frac{\sigma_{xy} + c_3}{\sigma_x\sigma_y + c_3} \tag{2.5}$$

The SSIM for each block combines comparative measures with weighted exponents:

$$\text{SSIM}(x,y) = l(x,y)\alpha \cdot c(x,y)\beta \cdot s(x,y)^\gamma$$

Selecting the third denominator stabilizing constant as

$$c_3 = c_2/2$$

simplifies combining the c and s components when exponents match ($\beta = \gamma$), because the numerator of c becomes twice the denominator of s, canceling to leave just a 2. Setting weights α, β, γ to 1 reduces the formula to the special case above. [27]

3.LPIPS

The Learned Perceptual Image Patch Similarity (LPIPS) metric measures how perceptually similar two images are. [28]

LPIPS works by comparing the feature activations of small patches from the two images using a pre-defined neural network. This approach aligns closely with human judgments of visual similarity. A lower LPIPS score indicates that the corresponding image patches appear more alike to human observers. [28]

Both input image patches should have the shape (N, 3, H, W), where the minimum dimensions of H and W depend on the selected backbone network. [28]

The LPIPS distance d between a reference image x and a distorted image y is computed as follows: [29]

$$d(x,y) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \left\| w_l \odot (\hat{x}^l_{h,w} - \hat{y}^l_{h,w}) \right\|_2^2 \tag{2.6}$$

- l: The specific layer in the neural network (often VGG, AlexNet, or SqueezeNet).
- $\hat{x}^l, \hat{y}^l$: The extracted feature activations for images x and y, which are unit-normalized across the channel dimension.
- H_l and W_l: The spatial height and width of the feature maps at layer l.
- h, w: The spatial coordinates for the pixels/units within the feature map.
- $\odot$: The channel-wise multiplication operator.
- w_l: A learned weight vector that scales the activations for each channel to better align with human perceptual judgments.

A lower score means the images are more perceptually similar, while a higher score indicates a larger perceptual difference.

2.4.4 SIFT: Scale-Invariant Feature Transform

Creating a paired image dataset, introduces some challenges such as image misalignment without the proper rigs. To combat this issue, several algorithms were implemented to filter through invalid pairs.

The scale-invariant feature transform (SIFT), a computer vision algorithm David Lowe developed in 1999, [30] identifies, characterizes, and connects local image details. Its uses encompass object recognition, robotic mapping and navigation, image stitching, 3D modeling, gesture recognition, video tracking, individual wildlife identification, and motion tracking. [31]

1. Detection of Scale-Space Extrema: This procedure begins with identifying points of interest, called keypoints within the SIFT framework. The image undergoes convolution with Gaussian filters at various scales, followed by computing the difference between successive Gaussian-blurred images. Keypoints appear as maxima or minima of the Difference of Gaussians (DoG) across scales. In particular, a DoG image $D(x,y,\sigma)$ is defined as: [31]

$$D(x,y,\sigma) = L(x,y,k_i\sigma) - L(x,y,k_j\sigma) \tag{2.7}$$

w$L(x,y,k\sigma)$ Convolution of the original image

$I(x,y)$ Original Image

The Gaussian blur $G(x,y,k\sigma)$

Gaussian Blur scale $k\sigma$, i.e.,

$$L(x,y,k\sigma) = G(x,y,k\sigma) * I(x,y) \tag{2.8}$$

A Difference-of-Gaussian (DoG) image between scales $k_i\sigma$ and $k_j\sigma$ results from subtracting Gaussian-blurred images at those scales. For scale space extrema

detection within the SIFT algorithm, convolution with Gaussian blurs occurs across varying scales. The resulting images are grouped by octave—defined as doubling σ—with k_i selected to maintain a consistent number of convolved images per octave. Subsequently, Difference-of-Gaussian images are derived from adjacent Gaussian-blurred images in each octave. [31]

After generating DoG images, keypoints appear as local minima or maxima across scales when each pixel is compared to eight neighbors at the same scale and nine in adjacent scales; if a pixel's value is the highest or lowest among all those examined, it qualifies as a candidate keypoint. [31]

2. Keypoint Localization: The extrema locations couldn't just be guessed, so a Taylor expansion of the scale-space function was used to interpolate the exact sub-pixel location of the maximum. Low-contrast points were then thrown out using a Hessian matrix to keep the data clean.

3. Orientation Assignment: To make sure matches survived rotations, SIFT calculates a dominant orientation based on local image gradients:

4. Descriptor Formulation: Finally, this produced a robust, 128-dimensional vector for every single keypoint, giving the relevant tools to match the data between the different image sensors.

2.4.5 RANSAC and Homography Estimation

Even with SIFT, there still hapened to be some outliers. To actually warp the 2D plane of the Fujifilm image to match the iPhone, a 3×3 Homography matrix H was needed.

RANSAC was ran to iteratively grab 4 random keypoint pairs, compute the matrix, and test it against the rest of our points. Matrices which resulted in the highest consensus were kept. Running this complex math over thousands of 4K images was quite taxing on single-core Python scripts, forcing the engineering of a heavy multi-processing architecture which is further explained in Chapter 4.

CHAPTER 3

METHODOLOGY: DATA ACQUISITION AND CONTROLS

3.1 Introduction

A core principle in machine learning is that a deep learning model is bottlenecked by the quality of the training data. Supervised Image-to-Image translation models demand paired datasets where the spatial geometry between the source and target is perfectly identical. Generating this paired dataset was challenging and there were several measures taken to ensure proper quality.

A paired image dataset contains strictly aligned input and output images (e.g., x_i and y_i), where every image represents the exact same scene across different styles, lighting conditions, or modalities. They are primarily used to train supervised models like Conditional GANs and diffusion transformers for image-to-image translation. [32] [33] [34] [35]

3.2 Image Acquisition

The way these images were captured was to ensure data diversity from pair to pair but also image alignment between pairs. No rig was used, but pictured were captured in the same equivalent focal lengths, time of day, location and conditions.

Getting the focal length right was especially challenging. Since Fujifilm JZ100 uses a CCD sensor, whereas Apple iPhone 17 Pro Max uses a CMOS sensor, focal lengths don't match up even if the milimeter count is the same. Thankfully, both cameras have a similar focal length range, with the Apple iPhone 17 Pro Max spanning from 24mm to 100mm and the Fujifilm 25mm-200mm. The iPhone also offers transitional digital "optical-quality" focal length using sensor crop, which unlock the 28mm, 35mm and 200mm focal lengths. [36] [37]

The image types were also important. The initial images were captured as JPEGs on Fujifilm and DNG on iPhone using *Adobe Project Indigo*. After capturing the iPhone photos HDR needed to be disabled to ensure proper training for the neural

networks. These images were converted to SDR range using *Adobe Lightroom* and exported as JPEG. This ensured that images were better paired.

The Fujfilm images were 4288×3216 in dimension and about 5.5 megabytes in size. The color profile of the Fujifilm images was sRGB and the color space RGB. The iPhone images were trickier to match, as by default they get processed in the P3 color profile. Converting them to sRGB was crucial to perfectly match the images. The size differed slightly with dimensions of 4032×3024 and an average size of 1.5 megabytes while for the sensor-cropped photos an average dimension of 3665×2749 and average size of 800 kilobytes.

3.3 Hardware and Sensor Specifications

This dataset was built by contrasting two fundamentally opposed camera architectures to capture the decade-long leap from legacy CCD to modern computational CMOS.

Specification	**Fujifilm JZ100 (Target)**	**iPhone 17 Pro Max (Source)**
Release Year	2012	2026
Sensor Architecture	Charge-Coupled Device (CCD)	Back-Illuminated (BSI) CMOS
Resolution	14 Megapixels	48 Megapixels
Data Output	8-bit highly compressed JPEG	16-bit linear DNG *(via Project Indigo)*
Dynamic Range	Low — immediate highlight clipping	High — multi-frame computational HDR

Table 3.1. Hardware Specifications of the Stereoscopic Capture Array

3.3.1 The Source Domain: iPhone 17 Pro Max

For the ground-truth ($\mathscr{X}$), an iPhone 17 Pro Max was used. This device houses a 48-megapixel back-illuminated (BSI) CMOS sensor. The architecture of a modern CMOS sensor allows for extremely rapid, parallel pixel readout, which is what makes computational photography possible in the first place. Its wide dynamic range and high

quantum efficiency made it the perfect candidate to establish it as a baseline.

3.3.2 The Target Domain: Fujifilm JZ100

The legacy target domain ($\mathscr{Y}$) was the 2012 Fujifilm JZ100. This compact point-and-shoot uses a 14-megapixel CCD sensor. Unlike CMOS sensors where every pixel has its own amplifier, a CCD sensor physically transports the accumulated charge across the chip to a single output node. It acts almost like a global shutter, which is great for minimizing rolling shutter artifacts, but terrible for electrical bottlenecks.

More importantly for this research, the JZ100 has a notoriously low full-well capacity. When pointed at bright scenes, i.e. a bright sky, the electron buckets immediately overflow, causing aggressive highlight clipping. The networks that will be trained need to learn how to emulate this pattern of data destruction.

3.4 Software Intervention: Bypassing the ISP

A key problem that occurs when creating an image dataset from a modern smartphone(especially with iPhone) is that the Image Signal Processor (ISP) processes every image automatically with no way of turning it off. Native smartphone camera apps rely heavily on computational photography introducing several key attributes to scenes: excessive edge sharpening, heavy local tone mapping, aggressive noise reduction, and semantic HDR merging. Therefore, using the native iPhone camera app was not going to work since it would be impossible to map the sensor's physics.

To circumvent the iPhone's aggressive ISP, *Adobe Project Indigo* was used. This software captured 32-frame RAW merges directly from the sensor buffer before the phone could apply its standard over-sharpening. By doing this, mathematically linear, 16-bit DNG (Digital Negative) files were obtained. This resulted in a pretty "clean" ground truth. The Fujifilm, on the other hand, only outputs highly compressed 8-bit JPEGs, which is okay in this case since the neural network need to learn the way the Fujifilm processes images(or doesn't).

3.5 Environmental Controls and Capture Protocol

Since the main goal of this thesis was for neural networks to learn the physical sensor difference, all the controllable variables had to be removed. Therefore, a process was introduced to capturing these images:

- **Lighting Diversity:** 61 pairs were shot within seconds of each other to maintain same conditions, mainly outdoors with a few indoor shots. Shooting in diverse lighting conditions was critical to ensure diversity in the dataset: harsh midday sun to intentionally force the CCD to clip, overcast diffuse lighting, and low-angle golden hour shots.
- **White Balance Locking:** Automatic White Balance (AWB) is a software trick that varies wildly between manufacturers. The white balance was manually locked on iPhone, whereas the Fujifilm was kept in auto white balance to let the camera make the choices, which the neural networks need to replicate.

3.6 Dataset Scaling and Augmentation

Processing 4K (3840×2160) full resolution images is quite a demanding task for any pre-processing pipeline, so to decrease compute time the following steps were taken:

1. **Tiling:** A script was written to crop the 4K images into tiny 256×256 pixel patches.
2. **Overlapping:** To ensure crucial structures weren't cut in half, the grid was ran with a 50% spatial overlap (a stride of 128 pixels).

By doing this, the dataset now consists of 24,565 256×256 patch pairs. Furthermore, it was split into train/test/validation roughly in the following ratios 80/10/10 resulting in 20,827 pairs for train, 3,179 pairs for test and 2,480 for validation with a size of 1.45 gigabytes.

CHAPTER 4

IMPLEMENTATION: SOFTWARE ENGINEERING AND PREPROCESSING

4.1 Introduction

One of the most important aspects of this thesis was getting the implementation right. Selecting the relevant technologies, optimizing the models to run and train in the respective systems and ensuring proper measurement and quality control of the process was crucial.

4.2 Parallel Preprocessing Pipeline

The raw image pairs of 4K images totaled about 417 Megabytes in size. Processing these images sequentially using standard techniques such as single-core Python scripts,it was going to take days—if not weeks—just to align and crop the images.This approach was completely invalid so a custom Hybrid Parallel Pipeline specifically optimized for Apple Silicon M1 Pro was designed.

4.3 The Compute Hardware: Apple Silicon M1 Pro

The entire dataset pre-processing phase was run on a 2021 MacBook Pro equipped with the Apple Silicon M1 Pro chip. What makes M1 Pro different from other systems is that it is a System on Chip(SoC) built around a Unified Memory Architecture (UMA).

In a traditional x86 PC build, the CPU and discrete GPU have entirely separate pools of memory, and they talk to each other across a PCIe bus. When manipulating massive arrays, a lot of time is wasted by just copying data back and forth between system RAM and VRAM. The M1 Pro, however, features a high-bandwidth, low-latency memory pool that both the CPU and the integrated GPU can access simultaneously.

4.4 Overcoming the Python GIL: A Hybrid Parallel Pipeline

Python has been named the industry standard in the Data Mining field, especially when dealing with pre-processing for neural networks. One thing that had to be overcomed was the Global Interpreter Lock (GIL).

The GIL is essentially a mutex that protects access to Python objects, physically preventing multiple native threads from executing Python bytecodes at the same time. While this makes Python easy to use, it bottlenecks CPU-bound tasks in modern multi-core environments. After sucessfully identifiying that the pipeline was not utilizing the full power of the system a Hybrid Parallel Pipeline was developed.

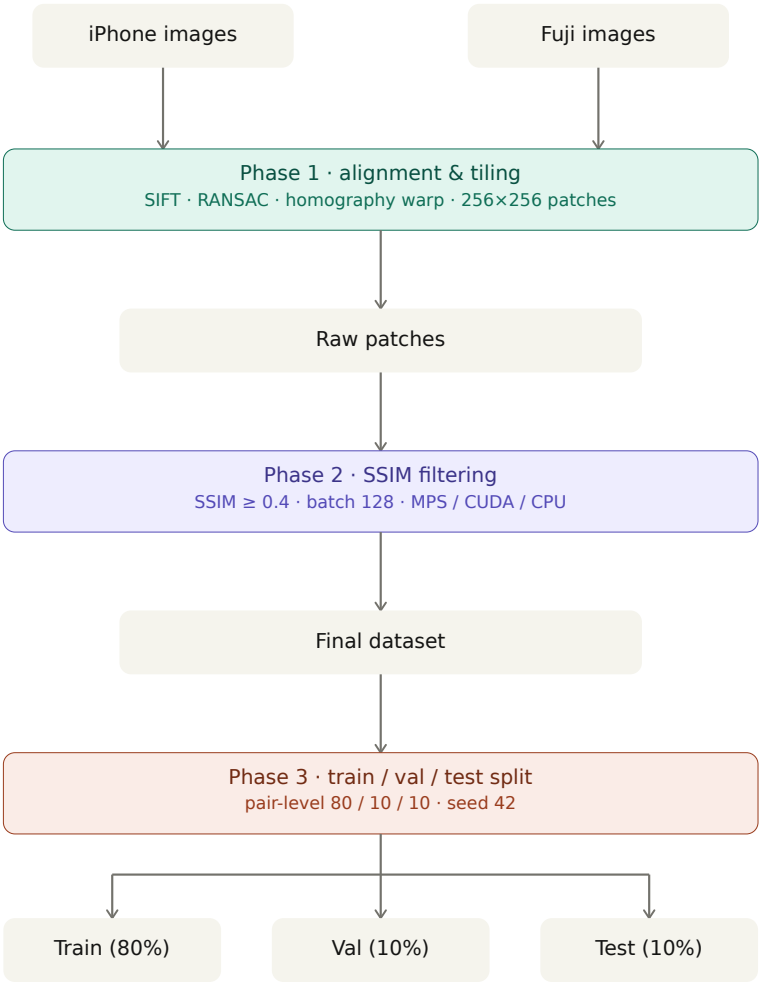

Figure 4.1. Preprocessing Pipeline for iPhone to Fuji Dataset

4.4.1 Multiprocessing for CPU-Bound Tasks

The feature extraction (SIFT) and geometric alignment (RANSAC homography estimation) are heavy, brutal mathematical operations. Because these tasks purely involve the CPU crunching massive 3840×2160 matrices, trying to use Python's threading library didn't impact the processing speed positively, since threads were forced to take turns by the GIL.

Getting around this issue required using Python's `concurrent.futures.ProcessPoolExecu` Instead of threading, this module actually forks the main Python interpreter into multiple completely independent processes. Each process gets its own separate memory space and its own GIL, allowing the operating system to finally schedule them concurrently across all of the M1 Pro's high-performance cores.

```
from concurrent.futures import ProcessPoolExecutor
import cv2

def align_image_pair(pair_path):
    source, target = load_images(pair_path)
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(source, None)
    kp2, des2 = sift.detectAndCompute(target, None)
    # Match descriptors and compute RANSAC Homography
    H = compute_homography(kp1, des1, kp2, des2)
    aligned = cv2.warpPerspective(source, H, (target.shape
       [1], target.shape[0]))
    return aligned

with ProcessPoolExecutor(max_workers=8) as executor:
    aligned_images = list(executor.map(align_image_pair,
       image_pairs))
```

Listing 4.1. Abstracted ProcessPoolExecutor implementation for Homography.

By setting the process pool to match the system's physical core count, the process hit near-linear scaling, reducing the SIFT/RANSAC alignment process from hours to minutes.

4.4.2 Multithreading for I/O-Bound Tasks

After successfully running the pipeline and generating the overlapping patches, another problem that had to be tackled was serializing those patches to the NVMe

solid-state drive as JPEG files. Writing to disk is an I/O-bound task, which means The CPU is idle, waiting for the SSD controller to acknowledge the write operation.

Because the Python GIL is temporarily released during I/O operations, spawning heavy, independent processes (which consume massive amounts of RAM) is actually a terrible idea for disk writing. Therefore the `concurrent.futures.ThreadPoolExecutor` was implemented.

```
from concurrent.futures import ThreadPoolExecutor

def write_patch_to_disk(patch, path):
    cv2.imwrite(path, patch)

with ThreadPoolExecutor(max_workers=32) as executor:
    for patch, path in zip(patches_to_write, paths):
        executor.submit(write_patch_to_disk, patch, path)
```

Listing 4.2. Abstracted ThreadPoolExecutor implementation for disk serialization.

A massive number of lightweight threads—far were created, exceeding our physical core count. This allowed the M1 Pro to keep the NVMe SSD queue depth continuously saturated, drastically accelerating the final dataset serialization.

4.5 Automated Quality Control: Structural Similarity Index (SSIM)

The issue thing that needed to be addressed is something called "ghosting", which appeared as a result of the slight timing discrepancy between the shooting of two pictures as well as the time differece between the rolling shutter of the CMOS and the global-like readout of the CCD. In outdoor environments, this meant that fast-moving elements like leaves blowing in the wind occupied completely different physical spaces in the source and target images.

Feeding these ghosted patches to the neural network would essentially make it quite challenging for the generator to learn a pattern between the photos, as the ground truth physically contradicted the input. Verifying the colossal number of patches manually was not an efficient method, so an automated Quality Control (QC) gate was engineered using the Structural Similarity Index (SSIM).

During generation, the script calculated the SSIM for every single 256×256 patch pair. A strict threshold was set: any pair dropping below an SSIM < 0.4 was immediately flagged as anomalous and permanently discarded from the dataset. This

automated programmatic filtration ensured the dataset was free from catastrophic homography failures and moving artifacts.

CHAPTER 5

MACHINE LEARNING AND RESULTS

5.1 Using MPS for training on Apple Silicon

As discussed previously, Apple Silicon differs from x86 systems, so model training demands distinct principles. On macOS, training occurs via Metal Performance Shaders (MPS), which PyTorch employs for GPU acceleration.

This MPS backend enhances PyTorch by offering scripts and tools to configure and execute operations on Mac. It boosts performance through kernels refined for each Metal GPU family. The mps device translates machine learning graphs and operations onto the MPS Graph framework and optimized kernels from MPS—ensuring strong results immediately. Developers may also use custom Metal kernels for greater adaptability. [38]

5.2 Models Compared and Hyperparameter Tuning

A comparison was a handy way to see how these two models would compare in VRAM consumption and batch-size optimization.

5.2.1 U-Net Baseline (Regressive)

The baseline was built using a standard U-Net architecture, training it exclusively with an L_1 (Mean Absolute Error) loss function. Since U-Net acts as a mathematical regressor, its only goal is to minimize the absolute difference between its generated output and the target CCD image.

Table 5.1. Empirical VRAM Overhead and Batch Size Optimization (M1 Pro)

Architecture	**Loss Function**	**Optimum Batch Size**	**Peak VRAM Usage**
U-Net	L_1 (MAE)	20	$\sim$11.7 GB
Pix2Pix	Adversarial + L_1	12	$\sim$10.8 GB

To maximize the learning rate across the 20827-patch dataset, the U-Net batch size was empirically optimized at 20. As Table 5.1 shows, anything larger than 20 put immense pressure on the VRAM capacity during the backpropagation of the encoder-decoder weights.

5.2.2 Pix2Pix (Conditional GAN)

For the adversarial model, Pix2Pix was used, pairing a U-Net Generator with a PatchGAN Discriminator. Because the training graph requires holding the weights, activations, and gradients for two distinct networks simultaneously in memory, the footprint of Pix2Pix was practically double that of the baseline U-Net.

5.3 Evaluation Metrics

Evaluation metrics were crucial for finding the best performing epoch per model. The evaluation metrics used consisted of: PSNR, SSIM and LPIPS. The highest PSNR score was used as the determining factor when picking the best epoch. An early stopping logic was implemented with a patience of 10 (train for 10 epochs after not exceeding PSNR to stop training).

5.3.1 Distortion vs Perceptual Metrics

PSNR (Peak Signal-to-Noise Ratio) and **SSIM (Structural Similarity Index)** were used to measure raw distortion, Higher PSNR means lower distortion.

To measure the realism and textures of the images, **LPIPS (Learned Perceptual Image Patch Similarity)** was used. For LPIPS, a lower score is better, indicating the image has realistic, convincing textures, even if the underlying pixels don't perfectly line up mathematically.

5.4 Key Findings

Table 5.2. Quantitative Evaluation Metrics on Validation Set

Model	**PSNR (dB)** ↑	**SSIM** ↑	**LPIPS** ↓
U-Net (Baseline)	16.27	0.715	0.524
Pix2Pix (cGAN)	**16.83**	**0.731**	**0.481**

As shown in Table 5.2, both models obtain similar PSNR values on the validation set (U-Net: 16.27 dB; Pix2Pix: 16.83 dB). Pix2Pix preserves slightly higher structural similarity (SSIM = 0.731) and attains a lower LPIPS (0.481), indicating better perceptual quality and structural similarity despite comparable PSNR. Qualitatively, U-Net outputs tend to be smoother and less textural, while Pix2Pix produces sharper micro-contrast that better matches human judgments of realism.

In conclusion, the Pix2Pix model achieved vastly superior LPIPS scores while still maintaining marginally better PSNR and SSIM scores. The adversarial loss successfully forced the Generator to "hallucinate" the sharpness and micro-contrast of the iPhone ground truth while embedding it deeply within the unique color science of the Fujifilm CCD. To the human eye, the Pix2Pix outputs were incredibly convincing, successfully bridging the gap between the modern computational domain and the legacy sensor.

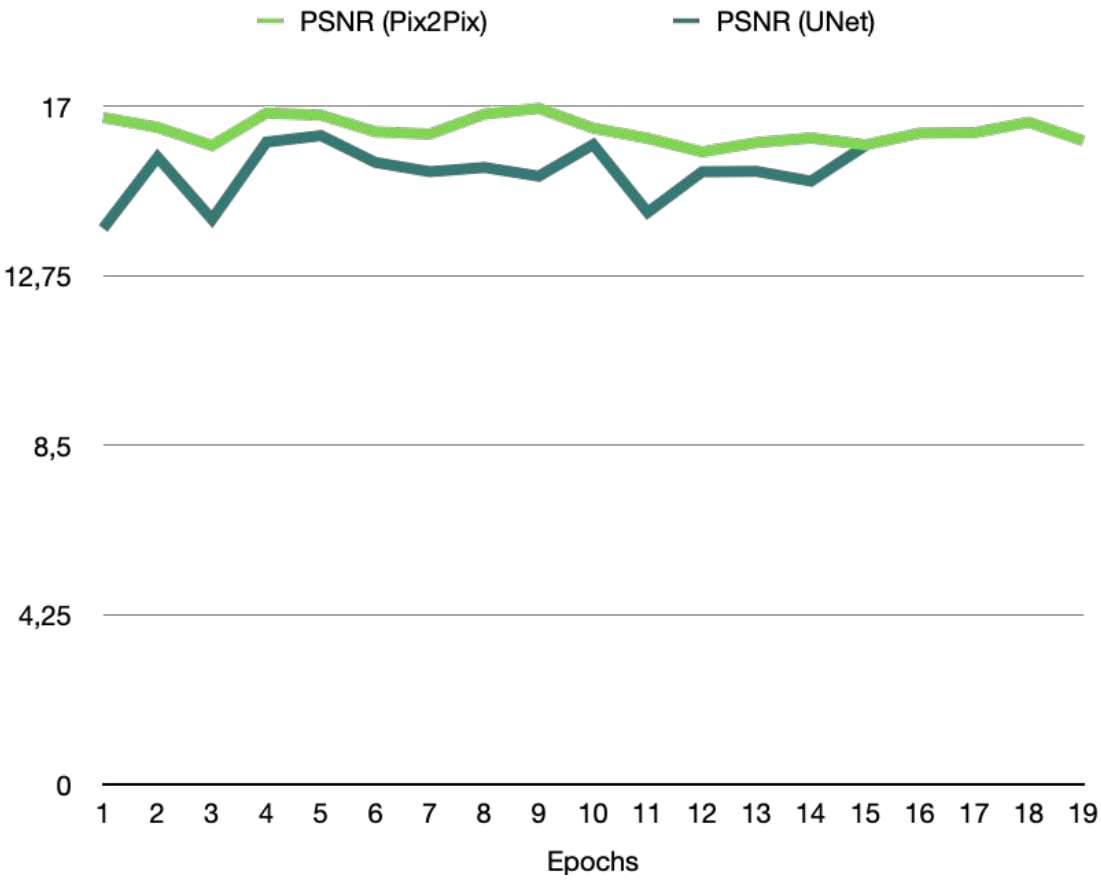

Figure 5.1. Validation PSNR across epochs for U-Net and Pix2Pix (higher is better).

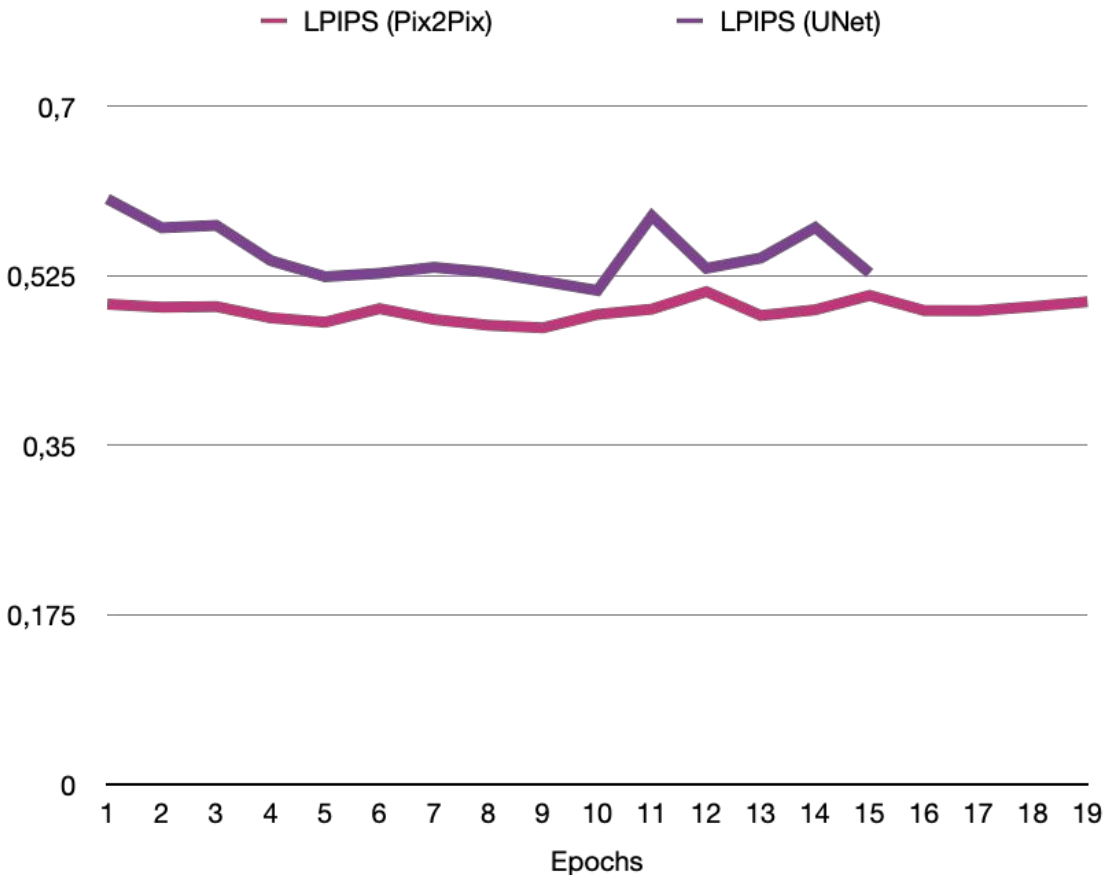

Figure 5.2. Validation LPIPS across epochs for U-Net and Pix2Pix (lower is better).

CHAPTER 6

CONCLUSION AND DISCUSSION

6.1 Synthesizing the Findings

The core question when creating this dataset was if a paired dataset would be enough for a deep learning architecture to successfully emulate the physical limitations of an aging, low-dynamic-range CCD sensor.

Looking at the test images the results indicate that a paired dataset is the only way to force a deep learning pattern to understand the differences between two distinct images and to map one's features to the other. As the scope of this thesis is mainly a study and outline of creating a paired image dataset and not fine-tuning deep learning models, the tuning and results of these models can be improved.

The evaluation metrics were helpful when selecting an interim best model for early stopping, but they do not tell the whole story. Since image processing, especially in cases as such where results are quite subjective, relies heavily on actual human input and validation, the evaluation metrics data isn't decisive for the model's performance.

After multiple test runs with different images, the results concluded that Pix2Pix was the superior model and that the paired dataset successfully enabled the model to learn the patterns from the image sets, as seen in the following comparisons.

Figure 6.1. Image Comparison Pix2Pix and Input.

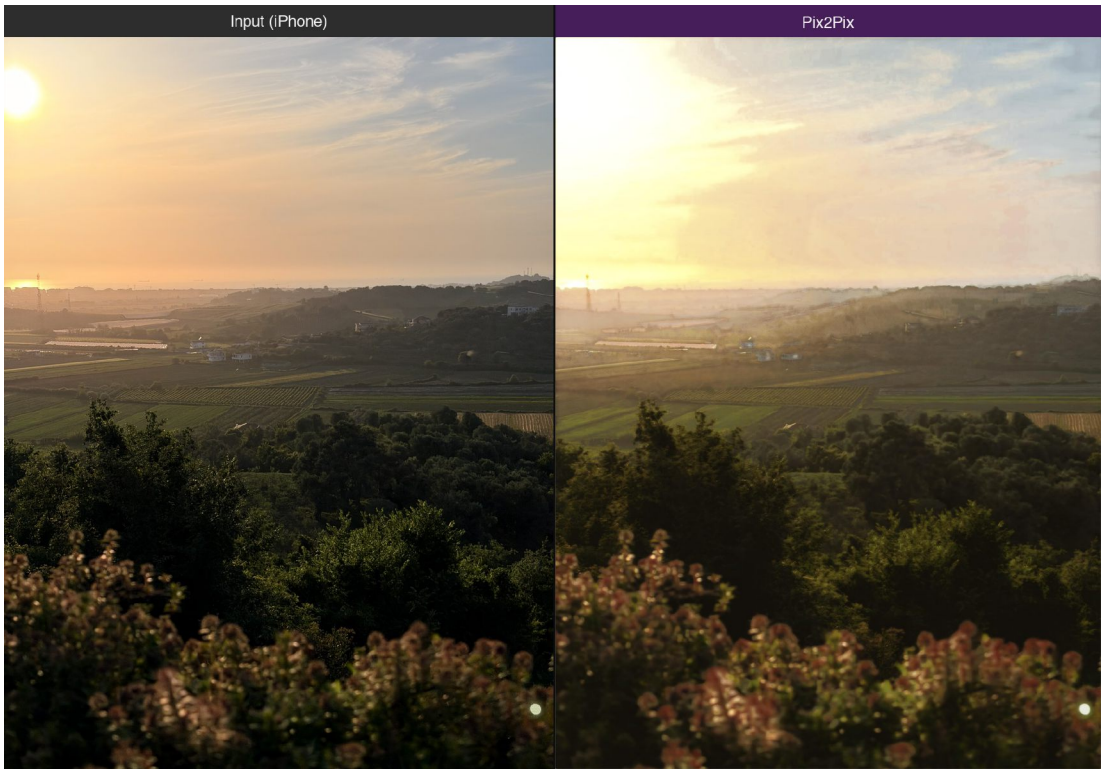

Figure 6.2. Image Comparison Pix2Pix and Input.

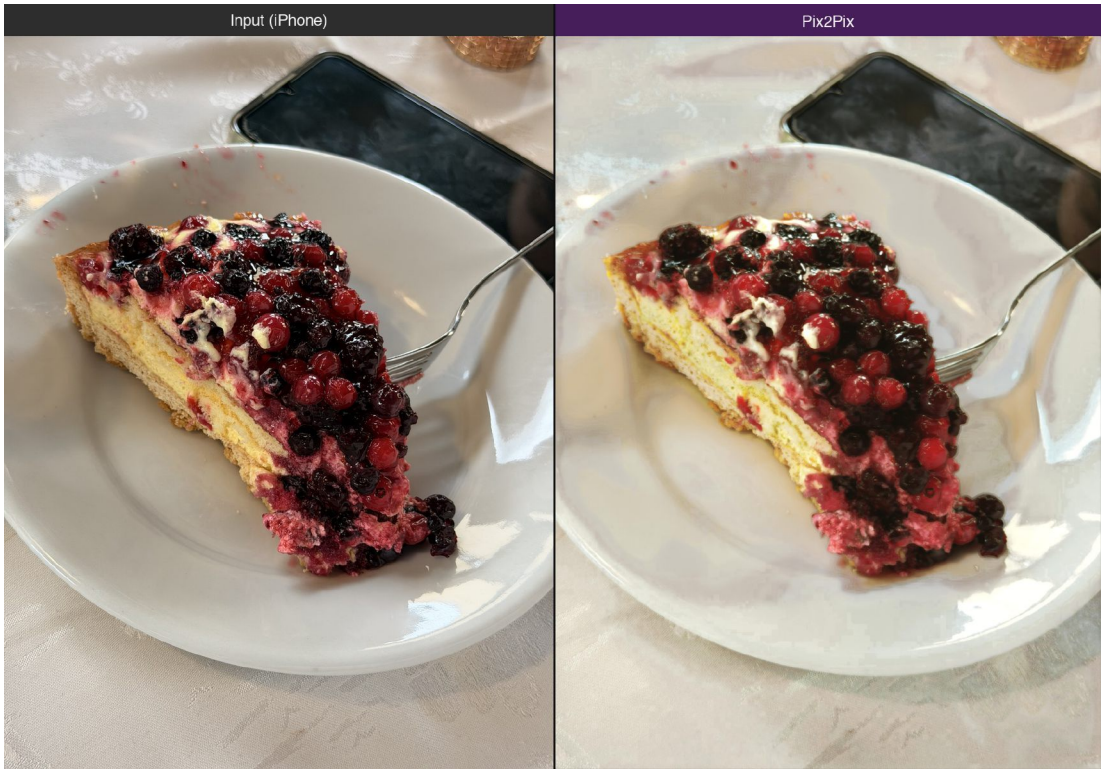

Figure 6.3. Image Comparison Pix2Pix and Input.

A case where the Pix2Pix model didn't do great is in Figure 6.1, where the Pix2Pix model didn't render the colors of the flower's petals in the right way, which means there is room for improvement in the AI tuning.

In Figure 6.2 is the most impressive result, where the model actually learned the pattern of deciding between blowing out the highlights in the sky or crushing the shadows. In this instance, detail was destroyed in the sky, successfully emulating the vintage sensor look.

The vintage look can also be seen clearly in Figure 6.3, where the model made great choices for highlight roll off, tone mapping and general feel of the photo.

As for the UNet model, the results told a different story. The model incorrectly processed all input images to look inverted and unappealing to the human eye. In this case, it is fair to say that UNet failed to learn the patterns in this paired image dataset.

Figure 6.4. Image Comparison UNet and Input.

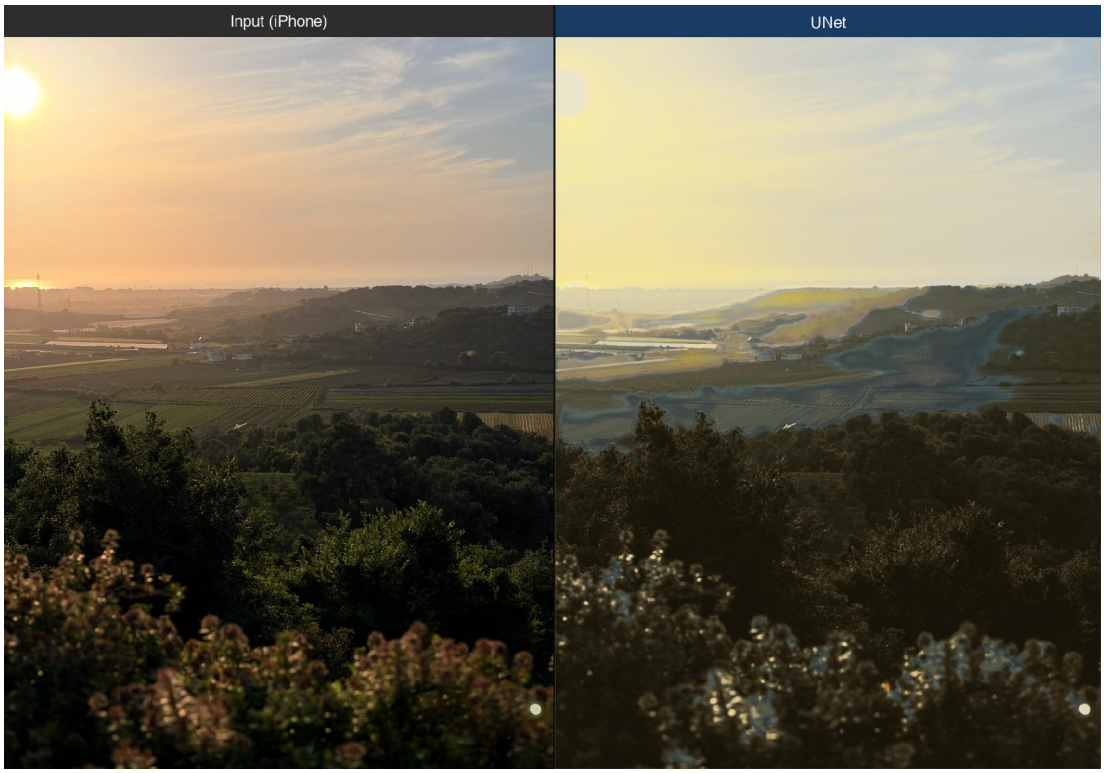

Figure 6.5. Image Comparison UNet and Input.

Figure 6.6. Image Comparison UNet and Input.

6.2 Addressing the Unsolved Problems

Even though most of the problems faced in this project were addressed there is still some room for improvement.

6.2.1 The Parallax Effect

Since photos were captured by hand and serially between devices there was inherently some differences between the images. Furthermore, it was not always possible to force the iPhone and the Fujifilm sensors to occupy the exact same physical space in the universe at the same time.

Even matching the focal lengths, the sensors are inherently different and the respective lenses and its composing elements bended the light differently which led to different kinds of distortion. Shooting most images at infinity focus, i.e. landscape shots, helped with alignment and the Parallax effect, but this limited the diversity of the dataset and hindered the scope of pattern recognition. This issue was mostly taken care of by the SSIM filtration threshold (discarding patches below 0.4 SSIM), but some parallax issues are impossible to avoid.

6.2.2 Aggressive CCD Clipping and Artifacts

The Fujifilm JZ100's CCD sensor has an incredibly low tolerance for highlights. Shoot a bright sky, and the sensor just throws its hands up, clipping entirely to pure white $(255, 255, 255)$.

This created a challenge for the neural network. The iPhone ground truth would have a beautifully rendered sky with gradients of blue and distinct cloud formations. The target domain was just a solid wall of white. From an information theory standpoint, this is an impossible mapping task. You are asking the network to learn a many-to-one function where highly distinct inputs all map to the exact same output. The Pix2Pix network frequently got confused in these regions, occasionally hallucinating weird edge artifacts where the clipping aggressively met a dark edge (like a building silhouette).

6.3 The Future of Sensor Emulation

Looking forward, there is a massive opportunity to expand this research beyond simple paired Image-to-Image translation. A way to truly eliminate the Parallax Error would be to use a bulky, expensive beam-splitter rig, allowing both sensors to share the exact same optical axis simultaneously. The next logical step would be to ditch paired datasets entirely. Another way to push this research forward would be pairing images with different digital cameras, not just one. Expanding the transformations to the iPhone image to truly capture that old sensor look and get the best from every digital camera. Another interesting study would be using RAW, straight from sensor, unprocessed iPhone images using *Halide Process Zero* or *Moment Camera Natural Processing*

Ultimately, this research proves that it is possible to emulate the aesthetic of legacy hardware on modern smartphone images. By leveraging adversarial networks and accepting the reality of the perception-distortion tradeoff, the physical limitations of vintage sensors can be mathematically deconstructed and replicated , bringing the organic flaws of early digital photography into the era of computational algorithms.

REFERENCES

[1] Computer Hope. What is an image? https://computerhope.com. Accessed: 2026-06-05.

[2] IBM Corporation. What is an image? — GDDM documentation. https://www.ibm.com/docs/bg/gddm?topic=ivu-what-is-image. Accessed: 2026-06-05.

[3] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018.

[4] Lumenci. Digital image processing: A beginner's guide. https://lumenci.com, 2023. Accessed: 2026-06-05.

[5] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2nd edition, 2022.

[6] WhiteWall. What is a digital image? https://whitewall.com, 2022. Accessed: 2026-06-05.

[7] Richard F. Haines and Sherry L. Chuang. The effects of video compression on acceptability of images for monitoring life sciences experiments. Technical Report NASA-TP-3239, A-92040, NAS 1.60:3239, NASA, 7 1992.

[8] Graham Hudson, Alain Léger, Birger Niss, István Sebestyén, and Jørgen Vaaben. Jpeg-1 standard 25 years: past, present, and future reasons for a success. *Journal of Electronic Imaging*, 27(4):040901, 8 2018.

[9] Joe Svetlik. The JPEG image format explained. Archived from the original on August 5, 2019.

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer International Publishing, 2015.

[11] Evan Shelhamer, Jonathan Long, and Trevor Darrell. Fully convolutional networks for semantic segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(4):640–651, 4 2017.

[12] Fatemeh Nazem, Fahimeh Ghasemi, Afshin Fassihi, and Alireza Mehri Dehnavi. 3D U-Net: A voxel-based method in binding site prediction of protein structure. *Journal of Bioinformatics and Computational Biology*, 19(2):2150006, 2021.

[13] Fatemeh Nazem, Fahimeh Ghasemi, Afshin Fassihi, and Alireza Mehri Dehnavi. A GU-Net-based architecture predicting ligand–protein-binding atoms. *Journal of Medical Signals & Sensors*, 13(1):1–10, 2023.

[14] Fatemeh Nazem, Fahimeh Ghasemi, Afshin Fassihi, and Alireza Mehri Dehnavi. Deep attention network for identifying ligand-protein binding sites. *Journal of Computational Science*, 81:102368, 2024.

[15] LMB Group. U-Net: Convolutional networks for biomedical image segmentation. https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/, 2026. Pattern Recognition and Image Processing, Department of Computer Science, Faculty of Engineering, University of Freiburg. Accessed: 2026-06-05.

[16] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[17] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training GANs. In *Advances in Neural Information Processing Systems*, volume 29, pages 2234–2242. Curran Associates, Inc., 2016.

[18] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1125–1134, 2017.

[19] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems*, volume 29, pages 4565–4573. Curran Associates, Inc., 2016.

[20] AI Summer. Vanilla GAN (GANs in computer vision: Introduction to generative learning). Archived from the original on June 3, 2020.

[21] Scribd User. Overview of Pix2Pix GAN Architecture. Scribd Presentation Document, 2026. Accessed: June 5, 2026.

[22] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE*

Conference on Computer Vision and Pattern Recognition (CVPR), pages 1125–1134, 2017.

[23] Medium Data Science Community. GAN pix2pix: Generative model for image-to-image translation. *Medium*, 2026. Accessed: June 5, 2026.

[24] Esri ArcGIS Developer Team. How Pix2Pix works, 2026. Accessed: June 5, 2026.

[25] Wikipedia contributors. Peak signal-to-noise ratio — Wikipedia, the free encyclopedia, 2026. [Online; accessed 5-June-2026].

[26] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.

[27] Wikipedia contributors. Structural similarity index measure — Wikipedia, the free encyclopedia, 2026. [Online; accessed 5-June-2026].

[28] TorchMetrics developers. Learned perceptual image patch similarity (LPIPS), 2026. Version 1.9.0. Accessed: 2026-06-05.

[29] Sara Ghazanfari, Siddharth Garg, Prashanth Krishnamurthy, Farshad Khorrami, and Alexandre Araujo. R-LPIPS: An adversarially robust perceptual similarity metric, 2023.

[30] David G. Lowe. Object recognition from local scale-invariant features. In *Proceedings of the Seventh IEEE International Conference on Computer Vision*, volume 2, pages 1150–1157. IEEE, 1999.

[31] Wikipedia contributors. Scale-invariant feature transform — Wikipedia, the free encyclopedia, 2026. [Online; accessed 5-June-2026].

[32] Qiang Zhu, Kuan Lu, Menghao Huo, and Yuxiao Li. Image-to-image translation with diffusion transformers and clip-based image conditioning. *arXiv preprint arXiv:2505.16001*, 2025.

[33] Shizuo Kaji and Satoshi Kida. Overview of image-to-image translation by use of deep neural networks: denoising, super-resolution, modality conversion, and reconstruction in medical imaging. *Radiological Physics and Technology*, 12(3):235–248, 2019. Code available at https://github.com/shizuo-kaji/PairedImageTranslation.

[34] Soumya Tripathy, Juho Kannala, and Esa Rahtu. Learning image-to-image translation using paired and unpaired training samples. *arXiv preprint arXiv:1805.03189*, 2018. Project page: https://tutvision.github.io/Learning-image-to-image-translation-using-paired-and-unpaired-training-sa

[35] Microsoft Applied Sciences. Imagepairs: A realistic super-resolution data set. *arXiv preprint arXiv:2004.08513*, 2020.

[36] Apple Inc. iPhone 17 Pro and 17 Pro Max - technical specifications, 2026. Accessed: 2026-06-05.

[37] Apple Inc. iPhone 17 Pro — pro fusion camera system, 2026. Technical Specifications and Camera Feature Overviews. Accessed: 2026-06-05.

[38] Apple Inc. Accelerated PyTorch training on Mac - Metal. https://developer.apple.com/metal/pytorch/, 2026. Accessed: 2026-06-07.

[39] Chris J. Solomon and Toby P. Breckon. *Fundamentals of Digital Image Processing: A Practical Approach with Examples in Matlab*. Wiley-Blackwell, 2010.

APPENDIX A

APPENDIX A: DATASET AND SOURCE CODE

This appendix provides a visual representation of the Neural Sensor Mapping dataset generated during this research, along with the core PyTorch implementation used for preprocessing, training, and evaluation.

A.1 Dataset Comparison

Figure A.1 demonstrates random sample patches extracted from the dataset. The top row illustrates the 14-megapixel Charge-Coupled Device (CCD) sensor of the Fujifilm JZ100, characterized by lower dynamic range and higher stochastic noise. The bottom row illustrates the perfectly aligned 48-megapixel CMOS sensor output from the iPhone 17 Pro Max, serving as the high-fidelity ground truth.

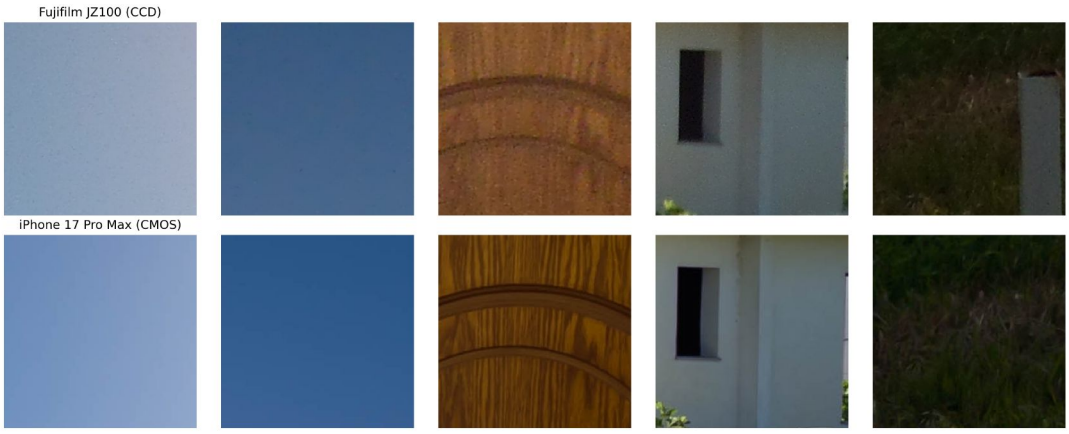

Figure A.1. Side-by-side comparison of 256x256 image patches. Top: Fujifilm JZ100 (CCD). Bottom: iPhone 17 Pro Max (CMOS).

A.2 Core Implementation Code

The following sections provide the complete source code for the experimental pipeline.

A.2.1 Dataset and Preprocessing (dataset.py)

This module handles the loading, normalization, and geometric augmentation of the stereoscopic image patches.

```
import os
from PIL import Image
import torch
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as transforms


class FujiIphoneDataset(Dataset):
    def __init__(self, root_dir, phase='train', crop_size
        =256):
        """
␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣root_dir␣(str):␣Root␣directory␣containing␣'train
    ',␣'val',␣and␣'test'␣subdirectories,
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣each␣with␣'fuji'␣and␣'iphone'␣
    folders.
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣Expected␣structure:
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣root_dir/train/fuji/
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣root_dir/train/iphone/
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣root_dir/val/fuji/
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣root_dir/val/iphone/
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣root_dir/test/fuji/
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣root_dir/test/iphone/
␣␣␣␣␣␣␣␣␣␣␣␣phase␣(str):␣'train',␣'val',␣or␣'test'
␣␣␣␣␣␣␣␣␣␣␣␣crop_size␣(int):␣Size␣of␣the␣crop␣to␣be␣applied␣
    if␣needed
␣␣␣␣␣␣␣␣"""
        self.root_dir  = root_dir
        self.phase     = phase
        self.crop_size = crop_size

        self.fuji_dir   = os.path.join(root_dir, phase, '
            fuji')
        self.iphone_dir = os.path.join(root_dir, phase, '
            iphone')

        if not os.path.isdir(self.fuji_dir):
```

```
            raise ValueError(f"Fuji directory not found: {
                self.fuji_dir}")
        if not os.path.isdir(self.iphone_dir):
            raise ValueError(f"iPhone directory not found: {
                self.iphone_dir}")

        self.image_filenames = sorted([
            f for f in os.listdir(self.fuji_dir)
            if f.lower().endswith(('.jpg', '.jpeg', '.png'))
        ])

        if len(self.image_filenames) == 0:
            raise ValueError(f"No image files found in {self
                .fuji_dir}")

        print(f"[{phase.upper()}] Found {len(self.
            image_filenames)} images.")

        self.normalize = transforms.Normalize((0.5, 0.5,
            0.5), (0.5, 0.5, 0.5))

    def __len__(self):
        return len(self.image_filenames)

    def __getitem__(self, idx):
        img_name = self.image_filenames[idx]

        fuji_path   = os.path.join(self.fuji_dir,   img_name
            )
        iphone_path = os.path.join(self.iphone_dir, img_name
            )

        fuji_img   = Image.open(fuji_path).convert('RGB')
        iphone_img = Image.open(iphone_path).convert('RGB')

        if self.phase == 'train':
            # Joint augmentation: same transform on both
                images
            if torch.rand(1) < 0.5:
                fuji_img   = fuji_img.transpose(Image.
                    FLIP_LEFT_RIGHT)
```

```
64                  iphone_img = iphone_img.transpose(Image.
                        FLIP_LEFT_RIGHT)
65              if torch.rand(1) < 0.5:
66                  fuji_img   = fuji_img.transpose(Image.
                        FLIP_TOP_BOTTOM)
67                  iphone_img = iphone_img.transpose(Image.
                        FLIP_TOP_BOTTOM)
68
69          fuji_tensor   = transforms.ToTensor()(fuji_img)
70          iphone_tensor = transforms.ToTensor()(iphone_img)
71
72          # Center-crop so both dimensions are divisible by 32
                (required by U-Net / PyNET)
73          _, h, w = fuji_tensor.shape
74          new_h   = h - (h % 32)
75          new_w   = w - (w % 32)
76
77          if new_h != h or new_w != w:
78              crop          = transforms.CenterCrop((new_h,
                    new_w))
79              fuji_tensor   = crop(fuji_tensor)
80              iphone_tensor = crop(iphone_tensor)
81
82          fuji_tensor   = self.normalize(fuji_tensor)
83          iphone_tensor = self.normalize(iphone_tensor)
84
85          return {'fuji': fuji_tensor, 'iphone': iphone_tensor
                , 'name': img_name}
86
87
88  def get_dataloaders(root_dir, batch_size=8, num_workers=4,
89                      pin_memory=False, persistent_workers=
                            False,
90                      prefetch_factor=2):
91      train_dataset = FujiIphoneDataset(root_dir, phase='train
            ')
92      val_dataset   = FujiIphoneDataset(root_dir, phase='val')
93
94      # prefetch_factor: each worker pre-loads this many
            batches into RAM ahead of time
95      # so the GPU never stalls waiting for data. Only valid
```

```
        when num_workers > 0.
    _prefetch = prefetch_factor if num_workers > 0 else None

    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=num_workers,
        drop_last=True,
        pin_memory=pin_memory,
        persistent_workers=persistent_workers and
           num_workers > 0,
        prefetch_factor=_prefetch,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
        pin_memory=pin_memory,
        persistent_workers=persistent_workers and
           num_workers > 0,
        prefetch_factor=_prefetch,
    )

    return train_loader, val_loader


def get_test_dataloader(root_dir, batch_size=1, num_workers
   =4):
    test_dataset = FujiIphoneDataset(root_dir, phase='test')
    test_loader  = DataLoader(
        test_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=num_workers,
    )
    return test_loader
```

Listing A.1. dataset.py

A.2.2 Pix2Pix Architecture (models/pix2pix.py)

The Conditional GAN architecture used to enforce high-frequency texture hallucination via the PatchGAN discriminator.

```
import torch
import torch.nn as nn
from .unet import UNet

class PatchGANDiscriminator(nn.Module):
    """Defines a PatchGAN discriminator"""

    def __init__(self, input_nc=3, ndf=64, n_layers=3, norm_layer=nn.BatchNorm2d):
        """Construct a PatchGAN discriminator
        Parameters:
            input_nc (int)  -- the number of channels in input images (real/fake + condition)
            ndf (int)       -- the number of filters in the first conv layer
            n_layers (int)  -- the number of conv layers in the discriminator
            norm_layer      -- normalization layer
        """
        super(PatchGANDiscriminator, self).__init__()
        kw = 4
        padw = 1
        sequence = [nn.Conv2d(input_nc, ndf, kernel_size=kw, stride=2, padding=padw), nn.LeakyReLU(0.2, True)]
        nf_mult = 1
        nf_mult_prev = 1
        for n in range(1, n_layers):  # gradually increase the number of filters
            nf_mult_prev = nf_mult
            nf_mult = min(2 ** n, 8)
            sequence += [
                nn.Conv2d(ndf * nf_mult_prev, ndf * nf_mult, kernel_size=kw, stride=2, padding=padw, bias=False),
                norm_layer(ndf * nf_mult),
                nn.LeakyReLU(0.2, True)
```

```
            ]

        nf_mult_prev = nf_mult
        nf_mult = min(2 ** n_layers, 8)
        sequence += [
            nn.Conv2d(ndf * nf_mult_prev, ndf * nf_mult,
               kernel_size=kw, stride=1, padding=padw, bias=
               False),
            norm_layer(ndf * nf_mult),
            nn.LeakyReLU(0.2, True)
        ]

        sequence += [nn.Conv2d(ndf * nf_mult, 1, kernel_size
           =kw, stride=1, padding=padw)]  # output 1 channel
            prediction map
        self.model = nn.Sequential(*sequence)

    def forward(self, input):
        """Standard␣forward."""
        return self.model(input)

class Pix2Pix(nn.Module):
    """
␣␣␣␣Wrapper␣for␣Pix2Pix␣that␣contains␣both␣Generator␣and␣
   Discriminator.
␣␣␣␣During␣training,␣we␣optimize␣them␣alternatively.
␣␣␣␣For␣inference,␣we␣just␣use␣the␣generator.
␣␣␣␣"""
    def __init__(self, in_channels=3, out_channels=3):
        super(Pix2Pix, self).__init__()
        # The generator is typically a UNet
        self.generator = UNet(n_channels=in_channels,
           n_classes=out_channels, bilinear=False)
        # The discriminator takes both the condition (input)
            and the generated/real image
        self.discriminator = PatchGANDiscriminator(input_nc=
           in_channels + out_channels)

    def forward(self, x):
        # Forward pass is just generator inference
        return self.generator(x)
```

Listing A.2. models/pix2pix.py

A.2.3 Training Loop (train.py)

The primary training script responsible for optimizing the Generator and Discriminator iteratively across the M1 Pro hardware.

```
import os
import argparse
import csv
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from tqdm import tqdm

from dataset import get_dataloaders
from metrics import MetricsCalculator
from models.unet import UNet
from models.pix2pix import Pix2Pix
from models.pynet import PyNET
from models.restormer import Restormer


def get_device():
    if torch.cuda.is_available():
        return torch.device('cuda')
    elif torch.backends.mps.is_available():
        return torch.device('mps')
    return torch.device('cpu')


def train():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset',         type=str,
       default='dataset', help='Dataset root path')
    parser.add_argument('--model',           type=str,
       choices=['unet', 'pix2pix', 'pynet', 'restormer'],
       required=True)
    parser.add_argument('--epochs',          type=int,
```

```
        default=50)
31      parser.add_argument('--batch_size',      type=int,
            default=8)
32      parser.add_argument('--lr',              type=float,
            default=2e-4)
33      parser.add_argument('--save_dir',        type=str,
            default='checkpoints')
34      parser.add_argument('--patience',        type=int,
            default=5,         help='Patience for early stopping'
            )
35      parser.add_argument('--num_workers',     type=int,
            default=4,         help='DataLoader workers')
36      parser.add_argument('--grad_accum',      type=int,
            default=1,         help='Gradient accumulation steps'
            )
37      parser.add_argument('--prefetch_factor', type=int,
            default=2,         help='Batches each worker pre-
            loads into RAM (2-4 recommended)')
38      parser.add_argument('--compile',         action='
            store_true',           help='torch.compile() the
            model')
39      parser.add_argument('--resume',          action='
            store_true',           help='Resume from the last
            saved checkpoint')
40      args = parser.parse_args()
41
42      os.makedirs(args.save_dir, exist_ok=True)
43
44      device  = get_device()
45      is_cuda = device.type == 'cuda'
46      use_amp = device.type in ('cuda', 'mps')
47
48      print(f"\n{''*50}")
49      print(f"  Device          : {device}")
50      print(f"  Training task   : iPhone  -> Fuji")
51      print(f"  Mixed precision : {'float16 v' if use_amp else
            'disabled'}")
52      print(f"  Batch size      : {args.batch_size}   (
            effective: {args.batch_size * args.grad_accum})")
53      print(f"  Workers         : {args.num_workers}   prefetch
            x{args.prefetch_factor}")
```

```
    print(f"  Grad accum      : {args.grad_accum}")
    print(f"  torch.compile   : {'will attempt v' if args.compile else 'off'}")
    print(f"{''*50}\n")

    #  DataLoaders
    train_loader, val_loader = get_dataloaders(
        args.dataset,
        batch_size=args.batch_size,
        num_workers=args.num_workers,
        pin_memory=is_cuda,
        persistent_workers=args.num_workers > 0,
        prefetch_factor=args.prefetch_factor,
    )

    #  Metrics + CSV log
    metrics_calculator = MetricsCalculator(device)

    csv_path = os.path.join(args.save_dir, f"{args.model}_training_log.csv")
    if not args.resume or not os.path.exists(csv_path):
        with open(csv_path, 'w', newline='') as f:
            csv.writer(f).writerow(['epoch', 'train_loss', 'val_psnr', 'val_ssim', 'val_lpips', 'lr'])

    #  Model
    model_cls = {'unet': UNet, 'pix2pix': Pix2Pix, 'pynet': PyNET, 'restormer': Restormer}
    model = model_cls[args.model]().to(device)

    if args.compile:
        try:
            model = torch.compile(model)
            print("torch.compile() active - first epoch slower while kernels compile.\n")
        except Exception as e:
            print(f"torch.compile() unavailable ({e}), continuing without it.\n")

    #  Optimizers + Schedulers
    if args.model == 'pix2pix':
```

```
89          optimizer_G = optim.AdamW(model.generator.parameters
               (),     lr=args.lr, betas=(0.5, 0.999),
               weight_decay=1e-4)
90          optimizer_D = optim.AdamW(model.discriminator.
               parameters(), lr=args.lr, betas=(0.5, 0.999),
               weight_decay=1e-4)
91          scheduler_G = optim.lr_scheduler.CosineAnnealingLR(
               optimizer_G, T_max=args.epochs, eta_min=args.lr *
                0.01)
92          scheduler_D = optim.lr_scheduler.CosineAnnealingLR(
               optimizer_D, T_max=args.epochs, eta_min=args.lr *
                0.01)
93      else:
94          optimizer = optim.AdamW(model.parameters(), lr=args.
               lr, weight_decay=1e-4)
95          scheduler = optim.lr_scheduler.CosineAnnealingLR(
               optimizer, T_max=args.epochs, eta_min=args.lr *
               0.01)
96
97      scaler = torch.cuda.amp.GradScaler() if is_cuda else
           None
98
99      best_psnr                   = -float('inf')
100     epochs_without_improvement = 0
101     start_epoch                 = 1
102
103     #  Resume
104     resume_path = os.path.join(args.save_dir, f"{args.model}
           _resume.pth")
105
106     if args.resume:
107         if not os.path.exists(resume_path):
108             print(f"␣␣No␣resume␣checkpoint␣found␣at␣{
                   resume_path}.␣Starting␣from␣scratch.\n")
109         else:
110             print(f"␣␣Resuming␣from␣{resume_path}")
111             ckpt = torch.load(resume_path, map_location=
                   device)
112
113             model.load_state_dict(ckpt['model_state_dict'])
114
```

```
115             if args.model == 'pix2pix':
116                 optimizer_G.load_state_dict(ckpt['
                        optimizer_G_state_dict'])
117                 optimizer_D.load_state_dict(ckpt['
                        optimizer_D_state_dict'])
118                 scheduler_G.load_state_dict(ckpt['
                        scheduler_G_state_dict'])
119                 scheduler_D.load_state_dict(ckpt['
                        scheduler_D_state_dict'])
120             else:
121                 optimizer.load_state_dict(ckpt['
                        optimizer_state_dict'])
122                 scheduler.load_state_dict(ckpt['
                        scheduler_state_dict'])
123
124             best_psnr                  = ckpt['best_psnr']
125             epochs_without_improvement = ckpt['
                    epochs_without_improvement']
126             start_epoch                = ckpt['epoch'] + 1
127
128             print(f"␣␣Resuming␣from␣epoch␣{start_epoch}␣␣|␣␣
                    best␣PSNR␣so␣far:␣{best_psnr:.4f}\n")
129
130     #  Epoch loop
131     try:
132         for epoch in range(start_epoch, args.epochs + 1):
133             model.train()
134             train_loss = 0.0
135
136             if args.model == 'pix2pix':
137                 optimizer_G.zero_grad(set_to_none=True)
138                 optimizer_D.zero_grad(set_to_none=True)
139             else:
140                 optimizer.zero_grad(set_to_none=True)
141
142             pbar = tqdm(enumerate(train_loader), total=len(
                    train_loader), desc=f"Epoch␣{epoch}/{args.
                    epochs}")
143
144             for step, batch in pbar:
145                 iphone = batch['iphone'].to(device,
```

```
                    non_blocking=is_cuda)
146                 fuji   = batch['fuji'].to(device,
                    non_blocking=is_cuda)
147
148                 if args.model == 'pix2pix':
149                     #  Discriminator step
150                     with torch.autocast(device_type=device.
                        type, dtype=torch.float16, enabled=
                        use_amp):
151                         fake_fuji = model.generator(iphone)
152
153                         pred_real = model.discriminator(
                            torch.cat([iphone, fuji],
                                      dim=1))
154                         pred_fake = model.discriminator(
                            torch.cat([iphone, fake_fuji.
                            detach()], dim=1))
155                         loss_D = (
156                             F.
                                binary_cross_entropy_with_logits
                                (pred_real, torch.ones_like(
                                pred_real)) +
157                             F.
                                binary_cross_entropy_with_logits
                                (pred_fake, torch.zeros_like(
                                pred_fake))
158                         ) * 0.5 / args.grad_accum
159
160                     (scaler.scale(loss_D) if scaler else
                        loss_D).backward()
161
162                     #  Generator step
163                     with torch.autocast(device_type=device.
                        type, dtype=torch.float16, enabled=
                        use_amp):
164                         pred_fake  = model.discriminator(
                            torch.cat([iphone, fake_fuji],
                            dim=1))
165                         loss_G_GAN = F.
                            binary_cross_entropy_with_logits(
                            pred_fake, torch.ones_like(
```

```
                pred_fake))
166             loss_G_pix = F.l1_loss(fake_fuji,
                    fuji) * 100.0
167             loss_G     = (loss_G_GAN +
                    loss_G_pix) / args.grad_accum
168
169         (scaler.scale(loss_G) if scaler else
                loss_G).backward()
170
171         if (step + 1) % args.grad_accum == 0:
172             if scaler:
173                 scaler.unscale_(optimizer_D)
174                 scaler.unscale_(optimizer_G)
175                 nn.utils.clip_grad_norm_(model.
                        discriminator.parameters(),
                        1.0)
176                 nn.utils.clip_grad_norm_(model.
                        generator.parameters(),
                        1.0)
177                 scaler.step(optimizer_D)
178                 scaler.step(optimizer_G)
179                 scaler.update()
180             else:
181                 nn.utils.clip_grad_norm_(model.
                        discriminator.parameters(),
                        1.0)
182                 nn.utils.clip_grad_norm_(model.
                        generator.parameters(),
                        1.0)
183                 optimizer_D.step()
184                 optimizer_G.step()
185             optimizer_D.zero_grad(set_to_none=
                    True)
186             optimizer_G.zero_grad(set_to_none=
                    True)
187
188         train_loss += loss_G.item() * args.
                grad_accum
189         pbar.set_postfix({
190             'D': f"{loss_D.item()␣*␣args.
                    grad_accum:.3f}",
```

```
                    'G':  f"{loss_G.item() * args.grad_accum:.3f}",
                    'px': f"{loss_G_pix.item():.3f}",
                })

            else:
                # Standard supervised (L1) training
                with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                    fake_fuji = model(iphone)
                    loss      = F.l1_loss(fake_fuji, fuji) / args.grad_accum

                (scaler.scale(loss) if scaler else loss).backward()

                if (step + 1) % args.grad_accum == 0:
                    if scaler:
                        scaler.unscale_(optimizer)
                        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                        scaler.step(optimizer)
                        scaler.update()
                    else:
                        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                        optimizer.step()
                    optimizer.zero_grad(set_to_none=True)

                train_loss += loss.item() * args.grad_accum
                pbar.set_postfix({'loss': f"{loss.item() * args.grad_accum:.4f}"})

        # LR scheduler step
        if args.model == 'pix2pix':
            scheduler_G.step()
            scheduler_D.step()
            current_lr = scheduler_G.get_last_lr()[0]
```

```
222             else:
223                 scheduler.step()
224                 current_lr = scheduler.get_last_lr()[0]
225 
226             #  Validation
227             model.eval()
228             val_psnr = val_ssim = val_lpips = 0.0
229 
230             with torch.no_grad():
231                 for batch in tqdm(val_loader, desc="␣␣
                        Validating", leave=False):
232                     iphone = batch['iphone'].to(device)
233                     fuji   = batch['fuji'].to(device)
234 
235                     with torch.autocast(device_type=device.
                            type, dtype=torch.float16, enabled=
                            use_amp):
236                         fake_fuji = model.generator(iphone)
                                if args.model == 'pix2pix' else
                                model(iphone)
237 
238                     fake_fuji = fake_fuji.float()
239                     p, s, l   = metrics_calculator.
                            evaluate_batch(fake_fuji, fuji.float
                            ())
240                     val_psnr  += p
241                     val_ssim  += s
242                     val_lpips += l
243 
244             N         = len(val_loader)
245             val_psnr  /= N
246             val_ssim  /= N
247             val_lpips /= N
248 
249             print(f"Epoch␣{epoch:3d}␣|␣lr␣{current_lr:.1e}␣|
                    ␣"
250                   f"PSNR␣{val_psnr:.4f}␣|␣SSIM␣{val_ssim:.4f
                        }␣|␣LPIPS␣{val_lpips:.4f}")
251 
252             #  Log to CSV
253             avg_train_loss = train_loss / len(train_loader)
```

```
254             with open(csv_path, 'a', newline='') as f:
255                 csv.writer(f).writerow([epoch,
                        avg_train_loss, val_psnr, val_ssim,
                        val_lpips, current_lr])
256
257             # Per-epoch model weights (for rollback)
258             torch.save(model.state_dict(), os.path.join(args
                    .save_dir, f"{args.model}_epoch_{epoch}.pth")
                    )
259
260             # Early stopping counters
261             if val_psnr > best_psnr:
262                 best_psnr                   = val_psnr
263                 epochs_without_improvement = 0
264                 torch.save(model.state_dict(), os.path.join(
                        args.save_dir, f"{args.model}_best.pth"))
265                 print(f"␣␣New␣best␣PSNR:␣{best_psnr:.4f}")
266             else:
267                 epochs_without_improvement += 1
268                 print(f"␣␣Early␣stop␣counter:␣{
                        epochs_without_improvement}/{args.
                        patience}")
269
270             # Resume checkpoint (saved AFTER counters
                    updated)
271             if args.model == 'pix2pix':
272                 resume_state = {
273                     'epoch':                        epoch,
274                     'model_state_dict':             model.
                            state_dict(),
275                     'optimizer_G_state_dict':
                            optimizer_G.state_dict(),
276                     'optimizer_D_state_dict':
                            optimizer_D.state_dict(),
277                     'scheduler_G_state_dict':
                            scheduler_G.state_dict(),
278                     'scheduler_D_state_dict':
                            scheduler_D.state_dict(),
279                     'best_psnr':                    best_psnr,
280                     'epochs_without_improvement':
                            epochs_without_improvement,
```

```
                }
            else:
                resume_state = {
                    'epoch':                        epoch,
                    'model_state_dict':             model.
                        state_dict(),
                    'optimizer_state_dict':         optimizer.
                        state_dict(),
                    'scheduler_state_dict':         scheduler.
                        state_dict(),
                    'best_psnr':                    best_psnr,
                    'epochs_without_improvement':
                        epochs_without_improvement,
                }
                torch.save(resume_state, resume_path)

                if epochs_without_improvement >= args.patience:
                    print("Early␣stopping␣triggered.␣Training␣
                        stopped.")
                    break

    except KeyboardInterrupt:
        if os.path.exists(resume_path):
            last = torch.load(resume_path, map_location='cpu
                ')['epoch']
            print(f"\n␣␣Training␣interrupted␣after␣epoch␣{
                last}.")
            print(f"␣␣To␣resume,␣add␣--resume␣to␣your␣
                command.")
        else:
            print(f"\n␣␣Training␣interrupted␣before␣any␣
                epoch␣completed␣␣nothing␣to␣resume␣from.")


if __name__ == '__main__':
    train()
```

Listing A.3. train.py